



INSTITUTO POLITÉCNICO NACIONAL

UPIITA - Unidad Profesional
Interdisciplinaria en Ingeniería y
Tecnologías Avanzadas IPN

Base de Datos Distribuidas

Tarea final de diseño de propuesta de
fragmentación

Marco González Luna 2024640044

Olvera Valdivia Cristobal 2024640014

Grupo

3TM2

Profesor

Carlos De La Cruz Sosa



upiita-ipn

Índice

Índice	2
1. Seleccionar uno de los 4 ejercicios	3
2. Grafo relacional	3
3. Algoritmo COM_MIN	4
Predicados Min-término (M)	5
4. Fragmentos derivados (SEMI-JOIN)	6
5. Validar el diseño de la fragmentación (Teoría de Tamer Ozsu)	7
6. Conclusión	8

1. Seleccionar uno de los 4 ejercicios

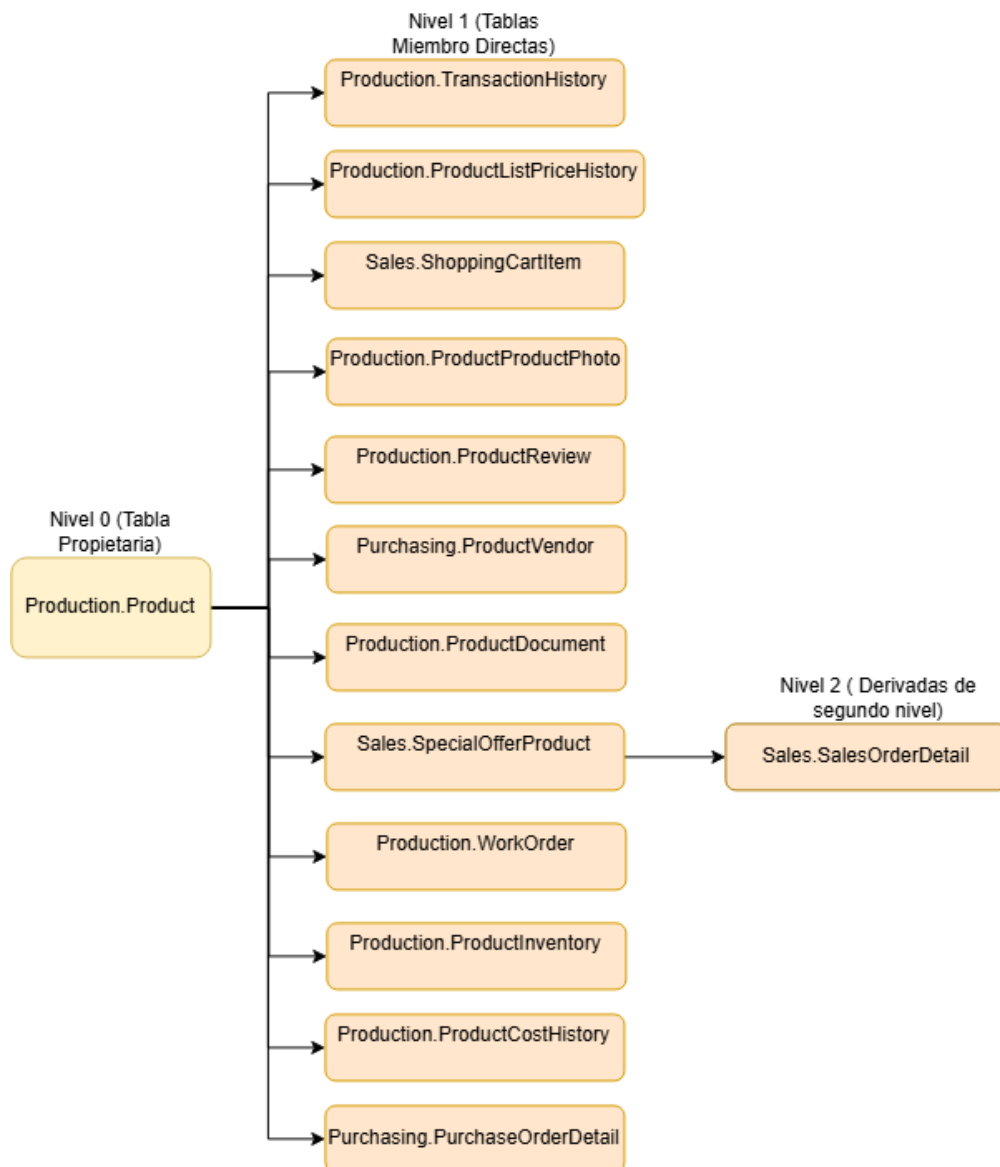
Ejercicio seleccionado: El ejercicio seleccionado por el equipo es el segundo.

Distribuir el catálogo masivo de productos en los 5 nodos regionales para optimizar las consultas de inventario local y evitar saturar un servidor central de manufactura.

Tabla propietaria: Production.Product

Atributo de fragmentación: ProductID

2. Grafo relacional



3. Algoritmo COM_MIN

Para lograr un balanceo de carga equitativo y evitar fragmentos vacíos debido a los saltos en la numeración de los IDs (504 productos reales), se calcularon los percentiles exactos en lugar de una división matemática simple. Cada predicado define una partición real no vacía y homogénea. En la teoría de Özsu, un predicado simple es relevante si divide el conjunto de datos de modo que las aplicaciones que acceden a cada nodo tengan un costo balanceado. Pr (predicados iniciales):

$$Pr = \{p_1, p_2, p_3, p_4, p_5, p_6, p_7, p_8\}$$

Se divide en 5 fragmentos ya que como indica se deben de separar en regiones que son: norte, sur, este, oeste, centro. Donde:

$$p_1: \text{ProductID} \leq 422$$

$$p_2: \text{ProductID} > 422$$

$$p_3: \text{ProductID} \leq 529$$

$$p_4: \text{ProductID} > 529$$

$$p_5: \text{ProductID} \leq 798$$

$$p_6: \text{ProductID} > 798$$

$$p_7: \text{ProductID} \leq 899$$

$$p_8: \text{ProductID} >$$

$$899$$

Se eliminan las negaciones redundantes de $(p_2, p_4, p_5, 8)$, ya que son el complemento de p_1, p_3, p_5, p_7 , quedan implícitas al construir los minitérminos. Solo sobreviven los predicados que aportan una distinción. El algoritmo COM_MIN define el siguiente conjunto de Pr' :

$$p_1: \text{ProductID} \leq 422$$

$$p_3: \text{ProductID} \leq 529$$

$$p_5: \text{ProductID} \leq 798$$

$$p_7: \text{ProductID} \leq 899$$

Predicados Min-término (M)

Un predicado min-término se forma evaluando conjunciones completas de cada predicado o su negación (p_i o $\bar{p}_i$). Debido a que los rangos son exhaustivos y mutuamente excluyentes, las combinaciones consistentes no vacías coinciden exactamente con cada rango regional:

$$m_1 = p_1 \equiv \text{ProductID} \leq 422$$

$$m_2 = \bar{p}_1 \wedge \bar{p}_3 \equiv 423 \leq \text{ProductID} \leq 529$$

$$m_3 = p_3 \wedge \bar{p}_5 \equiv 530 \leq \text{ProductID} \leq 798$$

$$m_4 = \bar{p}_5 \wedge p_7 \equiv 799 \leq \text{ProductID} \leq 899$$

$$m_5 = \bar{p}_7 \equiv \text{ProductID} > 899$$

$$\text{Product}_i = \sigma_{m_i}(\text{Production.Product}) \quad \text{para } i \in \{1,2,3,4,5\}$$

4. Fragmentos derivados (SEMI-JOIN)

Para garantizar que la información complementaria de cada producto resida en el mismo nodo físico, se aplica la operación *semi-join* ($\bowtie$) en cascada:

Derivación de Nivel 1 (para $i = 1,2,3,4,5$)

$$\text{TransactionHistory}_i = \text{TransactionHistory} \bowtie_{\text{ProductID}} \text{Product}_i$$

$$\text{ProductListPriceHistory}_i = \text{ProductListPriceHistory} \bowtie_{\text{ProductID}} \text{Product}_i$$

$$\text{ShoppingCartItem}_i = \text{ShoppingCartItem} \bowtie_{\text{ProductID}} \text{Product}_i$$

$$\text{ProductProductPhoto}_i = \text{ProductProductPhoto} \bowtie_{\text{ProductID}} \text{Product}_i$$

$$\text{ProductReview}_i = \text{ProductReview} \bowtie_{\text{ProductID}} \text{Product}_i$$

$$\text{ProductVendor}_i = \text{ProductVendor} \bowtie_{\text{ProductID}} \text{Product}_i$$

$$\text{ProductDocument}_i = \text{ProductDocument} \bowtie_{\text{ProductID}} \text{Product}_i$$

$$\text{SpecialOfferProduct}_i = \text{SpecialOfferProduct} \bowtie_{\text{ProductID}} \text{Product}_i$$

$$\text{WorkOrder}_i = \text{WorkOrder} \bowtie_{\text{ProductID}} \text{Product}_i$$

$$\text{ProductInventory}_i = \text{ProductInventory} \bowtie_{\text{ProductID}} \text{Product}_i$$

$$\text{ProductCostHistory}_i = \text{ProductCostHistory} \bowtie_{\text{ProductID}} \text{Product}_i$$

$$\text{PurchaseOrderDetail}_i = \text{PurchaseOrderDetail} \bowtie_{\text{ProductID}} \text{Product}_i$$

Derivación de Nivel 2 (en cascada, para $i = 1,2,3,4,5$)

La tabla de detalles de venta se fragmenta mediante la tabla intermedia SpecialOfferProduct:

$$\begin{aligned} &\text{SalesOrderDetail}_i \\ &= \text{SalesOrderDetail} \bowtie_{(\text{SpecialOfferID}, \text{ProductID})} \text{SpecialOfferProduct}_i \end{aligned}$$

5. Validar el diseño de la fragmentación (Teoría de Tamer Ozsu)

Regla 1: Completitud (*Completeness*)

- **Enunciado:** Si una relación R se descompone en $\{R_1, \dots, R_n\}$, toda tupla $t \in R$ debe encontrarse en al menos un fragmento R_i .
- **Validación Primaria:** La disyunción lógica de los predicados min-término cubre todos los valores enteros positivos posibles de ProductID:

$$\bigvee_{i=1}^5 m_i = (\text{ProductID} \leq 422) \vee (423 \leq \text{ProductID} \leq 529) \vee \dots \\ \vee (\text{ProductID} \geq 900) \equiv \text{TRUE}$$

- **Validación Derivada:** Por integridad referencial, toda tupla en una tabla hija posee un ProductID válido que existe en Product. Como Product es completo, cada tupla hija pertenece a su respectivo fragmento derivado.

Regla 2: Reconstrucción (*Reconstruction*)

- **Enunciado:** Debe existir un operador relacional ∇ tal que permita recuperar la relación original sin pérdida ni ganancia de información: $R = \nabla_{i=1}^n R_i$.
- **Validación:** Al tratarse de particionamiento horizontal, el operador de reconstrucción es la **Unión Relacional** ($\cup$). Como ejemplo:

$$\begin{aligned} \text{Product} &= \bigcup_{i=1}^5 \text{Product}_i \\ \text{ProductInventory} &= \bigcup_{i=1}^5 \text{ProductInventory}_i \\ \text{WorkOrder} &= \bigcup_{i=1}^5 \text{WorkOrder}_i \\ \text{SalesOrderDetail} &= \bigcup_{i=1}^5 \text{SalesOrderDetail}_i \end{aligned}$$

Regla 3: Disyunción (*Disjointness*)

- **Enunciado:** Si una tupla t pertenece al fragmento R_i , no debe pertenecer a ningún otro fragmento R_j ($i \neq j$). Es decir: $R_i \cap R_j = \emptyset$.
- **Validación Primaria:** Los intervalos definidos son mutuamente excluyentes:
$$m_i \wedge m_j = \text{FALSE} \quad (\forall i \neq j) \Rightarrow \text{Product}_i \cap \text{Product}_j = \emptyset$$
- **Validación Derivada:** Dado que una tupla en una tabla dependiente referencia a un único ProductID, y cada ProductID reside en un único fragmento primario, no existe duplicidad de tuplas entre fragmentos derivados:

$$M_i \cap M_j = \emptyset \quad (\forall i \neq j)$$

6. Conclusión

El diseño propuesto aborda directamente el problema de saturación del servidor central mediante una distribución horizontal óptima. Al basar los rangos del catálogo en la densidad real de los datos en lugar de rangos aritméticos ciegos, garantizamos que los cinco nodos procesen exactamente la misma carga de trabajo (aproximadamente 100 productos por servidor). Además, la aplicación rigurosa de fragmentación derivada en cascada (hasta el Nivel 2) cumple con el principio de localidad de datos: todas las consultas de stock, histórico de precios y órdenes de compra se pueden resolver íntegramente de manera local, eliminando los costosos cruces de información a través de la red distribuida.